PROJECT TITLE

AI-Powered Conversational Assistant with Local LLM Integration

PROJECT SUMMARY

Designed and developed a responsive, full-stack chatbot application that leverages a locally hosted Large Language Model (LLM) to provide intelligent and context-aware responses. The project aims to deliver a seamless conversational experience while ensuring data privacy by processing all requests locally without relying on third-party cloud APIs. The application features a modular architecture, consisting

of a Python/Flask backend integrated with LangChain and Ollama, and a modern, vanilla JavaScript frontend with real-time response streaming and a versatile embeddable widget.

TECHNOLOGY STACK

* Backend Framework: Python, Flask

* AI / LLM Framework: LangChain, Ollama

* Language Model: Qwen3:1.7b (Local Execution)

* Frontend: HTML5, CSS3, Vanilla JavaScript (ES6+)

* APIs & Networking: RESTful APIs, Flask-CORS, Server-Sent Events / Streams

KEY FEATURES & ACCOMPLISHMENTS

1. Local LLM Integration via LangChain and Ollama

- Integrated the Qwen3:1.7b local model using Ollama to handle natural language understanding and generation, ensuring zero data leakage to external servers.
- Built an intelligent query pipeline using LangChain's ChatPromptTemplate and StrOutputParser, streamlining the AI interaction flow.

2. Real-Time Streaming Responses

- Engineered a real-time text generation pipeline using Python generators and Flask's Response streaming capabilities.
- Designed the JavaScript frontend to consume the streamed data chunk-by-chunk, providing a fluid, ChatGPT-like typing experience for the end-user with minimal perceived latency.

3. Embeddable Chat Widget Architecture

- Developed a standalone embeddable chat widget (`widget.js` and `widget_snippet.html`) allowing the chatbot to be seamlessly integrated into any third-party website using a simple JavaScript snippet.
- Implemented Cross-Origin Resource Sharing (CORS) on the backend to securely handle requests from diverse origins hosting the widget.

4. Session Memory & Chat History

- Implemented a lightweight, robust session memory management system storing context across user interactions using unique session IDs.
- Seamlessly integrated history context with LangChain payloads natively, empowering the local LLM to deliver continuous, context-aware conversational threads safely without relying on external databases.

5. Modern and Responsive User Interface

- Designed a clean, intuitive, and highly responsive user interface using HTML5 and custom CSS3
- Ensured cross-device compatibility, offering an optimal chat experience on both desktop and mobile platforms.

SYSTEM ARCHITECTURE & WORKFLOW

1. Client-Side (Frontend):

- The user inputs a message in the chat interface or the embedded widget.
- The JavaScript client captures the input, sanitizes it, and dispatches a POST request to the Flask backend's `/chat` endpoint.
- The fetch API processes the streamed response body, dynamically updating the DOM to simulate real-time typing.

2. Server-Side (Backend):

- The Flask server receives the incoming JSON payload and extracts the message.
- The message is fed into the LangChain pipeline, which combines it with a pre-defined system prompt ("You are a helpful AI assistant").
- The formatted prompt is sent to the locally running Ollama service.

3. AI Generation & Delivery:

- Ollama processes the prompt using the Qwen3:1.7b model and yields text chunks as they are generated.
- The Flask generator function yields these chunks back to the client immediately, establishing a continuous data stream until the generation is complete.

CHALLENGES OVERCOME

* Latency Management: Initially, waiting for the entire LLM response before sending it to the client caused significant delays. I solved this by refactoring the backend to use LangChain's `.stream()` method and Flask's generator-based responses, successfully reducing Time-To-First-Token (TTFT) by over 80%.

* Cross-Origin Complexities: Building the embeddable widget introduced CORS errors when testing on different ports. I successfully configured `Flask-CORS` to securely accept cross-origin requests, ensuring the widget functioned reliably across various deployment environments.

* Frontend Stream Parsing: Handling raw data streams in vanilla JavaScript required careful implementation of the Streams API (`response.body.getReader()`) to ensure unicode characters and multi-byte text chunks were decoded and rendered flawlessly without visual glitches.

FUTURE ENHANCEMENTS

* Retrieval-Augmented Generation (RAG): Integrate a vector database (e.g., ChromaDB) to allow the chatbot to search and reference external documents or personal knowledge bases before generating an answer.

* Multi-Model Support: Add a dropdown in the UI to dynamically switch between different local models (e.g., Llama 3, Mistral) based on the user's specific needs or hardware capabilities.

PERSONAL LEARNINGS & IMPACT

This project served as a comprehensive deep-dive into the modern AI engineering stack. It bridged the gap between traditional web development (Flask, JS, CSS) and cutting-edge AI orchestration (LangChain, Local LLMs). By prioritizing local execution, I gained invaluable experience in optimizing performance for resource-constrained environments and handling real-time data streams over HTTP. The creation of the embeddable widget further strengthened my understanding of modular frontend design and secure API exposure. Ultimately, this project demonstrates my ability to architect, build, and deliver complete, end-to-end AI software solutions.